BOOKMAte Complete Interview Prep

Project: Spoiler-Safe Reading Companion / Agentic Temporal RAG

One-line pitch: This project answers questions about a book using only the portion the reader has already read. It uses temporal retrieval, a spoiler firewall, LangGraph orchestration, citation validation, and hybrid search to keep future text out of the LLM prompt.

1. What This Project Is

- A reading companion that lets users ask questions while reading a book.
- A RAG system because it retrieves relevant chunks and gives them to an LLM.
- A temporal RAG system because retrieval is constrained by the reader's current word position.
- An agentic AI project because it uses multiple coordinated steps: analyze, retrieve, assess, answer, validate, retry, refuse, or surface error.
- A spoiler-safe system because the LLM is not trusted with future text; the architecture prevents future text from entering the prompt.

Interview answer:

This is an agentic temporal RAG project. It combines retrieval augmented generation with a reader-position constraint. The key idea is that spoiler safety is enforced by the retrieval architecture, not only by prompting the LLM not to spoil.

2. Complete Architecture

```
React Frontend
  |
  | HTTP requests
  v
FastAPI Backend
  |
  | book upload, ingestion, session, chat APIs
  v
Ingestion Pipeline
  |
  | parse -> clean -> section -> chunk -> embed -> persist
  v
SQLite + FTS5 + NumPy Vector Matrices
  |
  | safe retrieval by reader position
  v
LangGraph Workflow
  |
  | analyze -> retrieve -> assess -> answer -> validate -> final/refuse/error
  v
LLM Provider: stub / Gemini / Groq / Ollama
```

Main layers:

Layer	Files	Responsibility
Frontend	web/src/App.tsx, web/src/api.ts	User interface, book upload/search, page slider, questions, answer display, citations.
API	app/api.py	FastAPI routes for books, sessions, upload, ingest, chat, API keys.
Ingestion	app/ingestion/*	Parse books, clean text, split into sections, chunk, embed, save.
Storage	app/store/db.py, app/store/vectors.py	SQLite metadata/chunks/FTS5 plus .npy embedding matrices.
Retrieval	app/retrieval/firewall.py, hybrid.py	Spoiler-safe vector + keyword retrieval and fusion.
Oracle	app/retrieval/oracle.py	Checks whether answer appears later, returns only boolean.
Agents	app/agents/*	Analyzer, answer prompt/model call, validator, provider wrapper.
Graph	app/graph.py	LangGraph orchestration with branches, retries, and final outcomes.

3. Upload and Ingestion Flow

```
User uploads TXT / EPUB / HTML
  -> /books/upload
  -> validate file type and size
  -> save raw file in data/raw
  -> create ingest job
  -> background worker runs ingest_file()
  -> parse file to plain text
  -> clean text
  -> split into chapters/sections
  -> chunk into sentence-aware overlapping chunks
  -> save chunks in SQLite
  -> embed chunk text
  -> save matrix as data/db/vectors/{book_id}.npy
```

Supported uploads: .txt, .epub, .html, .htm, .xhtml. PDF is deliberately not supported because extraction can scramble text order and make word offsets unreliable.

Why background jobs?

Indexing a whole book can take minutes. The API returns a job id immediately and the frontend polls the job status instead of holding one long request open.

4. Chunking in This Project

Chunking is handled in app/ingestion/chunker.py. The project cuts the full book once during ingestion, before any reader position is known. Boundaries must stay stable as the reader advances.

Setting	Value	Meaning
CHUNK_WORDS	320	Target chunk size in words.
CHUNK_OVERLAP	40	Approximate overlap between neighboring chunks.
Boundary style	Sentence-aware	Chunks avoid starting or ending mid-sentence.
Ordering	Narrative/global	idx, word_start, and word_end are monotonic across the whole book.

Each chunk stores id, book_id, idx, text, chapter, chapter_title, word_start, and word_end. For spoiler safety, a chunk is eligible only if word_end <= reader_position.

5. General RAG Chunking

- Fixed-size chunking: split every N tokens or characters.
- Sentence-aware chunking: keep chunks readable and avoid broken sentences.
- Paragraph/section chunking: preserve document structure.
- Recursive chunking: try section, then paragraph, then sentence, then token boundaries.
- Semantic chunking: split when topic meaning changes.
- Code-aware chunking: split by files, classes, functions, or symbols.

The usual tradeoff: chunks must be small enough for precise retrieval but large enough to include context. This project also needs chunks to preserve narrative order for spoiler safety.

6. Temporal Spoiler Firewall

> This is the most important part of the project. The firewall ensures future text does not reach the prompt.

```
reader_position = current word offset
n_p = count(chunks where word_end <= reader_position)

Safe vector search:
  matrix = vectors.prefix(book_id, n_p)
  search only matrix rows E[:n_p]

Safe keyword search:
  SELECT ... WHERE chunks_fts MATCH ?
             AND c.book_id = ?
             AND c.idx < n_p
```

Vector retrieval is structurally bounded: future rows are absent from the matrix operand. Keyword retrieval is predicate bounded: SQL must include c.idx < n_p.

Interview answer:

The spoiler firewall is not just a prompt instruction. For vector search, it physically slices the embedding matrix to only include chunks behind the reader. For keyword search, it applies the same prefix bound in SQL. Then it asserts safety before evidence reaches the prompt.

7. Retrieval Design

```
question
  -> vector search over safe prefix
  -> keyword search over safe FTS5 rows
  -> Reciprocal Rank Fusion combines ranked lists
  -> add neighboring safe chunks for context
  -> sort evidence in narrative order
  -> send to answer agent
```

Hybrid retrieval is used because vector search is good for semantic similarity, while keyword search is good for names, exact terms, and rare phrases. Reciprocal Rank Fusion combines rankings without needing score normalization.

8. LangGraph Workflow

```
START
  -> analyze_question
  -> retrieve
  -> assess_evidence
       if sufficient -> answer
       if thin       -> spoiler_oracle
  -> answer
  -> validate
       if grounded       -> finalise
       if bad citations  -> answer retry
       if insufficient   -> spoiler_oracle
       if provider error -> surface_error
  -> END
```

LangGraph is used because the system needs typed state, conditional branches, terminal outcomes, and a bounded retry loop. LangChain alone could provide prompts/model abstractions, but this workflow is naturally a graph.

9. Agents and Non-Agent Components

Component	Agent?	Role
Analyzer	Yes/lightweight	Classifies question intent for display: character, event, motivation, relationship, summary, clue, prediction.
Answer agent	Yes	Reads safe evidence and writes cited answer or INSUFFICIENT.
Validator	Yes/optional LLM	Checks citations and grounding; deterministic checks always run.
SafeRetriever	No	Safety infrastructure that retrieves only eligible evidence.
Spoiler oracle	No	Looks at future embeddings but returns only True/False, never future text.
LLM wrapper	No	Provider abstraction for Gemini, Groq, Ollama, and stub.

10. Answer Generation and Validation

The answer prompt tells the model to use only supplied evidence, cite every factual sentence, avoid speculation, and reply INSUFFICIENT if evidence does not support an answer.

Validation has two layers:

- Deterministic citation validation: every cited chunk id must be in the allowed evidence set.
- Optional model validation: checks whether claims are supported by cited passages.

Provider errors are not treated as refusals. The graph returns outcome error so outages do not masquerade as spoiler-safe behavior.

11. Data Stores

Store	What it contains	Why it matters
SQLite books	Book metadata, title, author, total words, chunk count	Lists indexed books and page mapping.
SQLite chunks	Full chunk text plus word offsets	Source of evidence and spoiler bounds.
SQLite chunks_fts	FTS5 keyword index	Keyword half of hybrid retrieval.
SQLite sessions	Reader position per session	Tracks what the reader has reached.
SQLite conversations	Questions, answers, outcomes, prompt hashes	Audit/debug history.
.npy vector matrix	One embedding row per chunk	Fast vector search and prefix slicing.

12. API Endpoints to Know

Endpoint	Purpose
GET /health	Backend health, indexed books, provider, embedding model.
GET /books	List indexed books.
POST /books/resolve	Search Project Gutenberg candidates.
POST /books/ingest	Start Gutenberg ingestion job.
POST /books/upload	Upload user-owned DRM-free book and start indexing.
GET /books/ingest/{job_id}	Poll background ingest status.
POST /sessions	Create reader session.
PATCH /sessions/{id}/position	Update reader position or page.
POST /chat	Ask a question at the current reader position.
POST/DELETE /config/api-key	Set or clear provider API key.

13. Testing Strategy

- test_firewall.py verifies prefix bounds and includes mutation-style safety tests.
- test_graph.py checks answered/refused outcomes, prompt payload safety, and citation subsets.
- test_ingestion.py checks chunk order, word_end bounds, and monotonic offsets.
- test_retrieval.py checks retrieval behavior.
- test_citations.py and test_spoilers.py check grounding and spoiler scenarios.
- The stub LLM lets the workflow run offline without an API key.

Interview answer:

The tests focus on the safety properties, not just happy paths. A key test proves that if the keyword prefix predicate is removed, the suite fails. That makes the firewall test meaningful.

14. Why Not Just Prompt the LLM?

Prompting alone is not enough because a model can ignore instructions, infer future facts from memory, or accidentally receive future text through retrieval. This project reduces risk by ensuring future text is not included in the prompt at all.

Important limitation: the system cannot completely stop parametric leakage, where a model already knows a famous book from training. It can prevent prompt leakage, and it measures parametric leakage separately.

15. Interview Questions and Strong Answers

What is this project?

It is an agentic temporal RAG reading companion that answers questions from only the part of a book the reader has already read.

Why is it temporal RAG?

Because retrieval is constrained by the reader's progress. The safe corpus changes over time as the reader advances.

Why is it agentic?

It uses a graph workflow with multiple specialized steps: analyzer, retriever, answerer, validator, oracle routing, retries, and error handling.

Why LangGraph?

The workflow needs branching, state, terminal outcomes, and retry loops. LangGraph expresses that more clearly than a linear LangChain chain.

What is the main innovation?

Spoiler safety is enforced architecturally. Future chunks are excluded from retrieval before the LLM sees evidence.

How are books chunked?

Books are parsed to plain text, cleaned, split into sections, then chunked into roughly 320-word sentence-aware chunks with 40-word overlap.

How does vector retrieval stay safe?

The embedding matrix is ordered by chunk index, and retrieval searches only E[:n_p], where n_p is the count of chunks fully behind the reader.

How does keyword retrieval stay safe?

The FTS5 query includes c.idx < n_p, so only already-read chunks can match.

What does the oracle do?

It checks future embeddings to decide if a question is likely answered later, but returns only a boolean and never future text.

What would you improve?

I would add reranking, improve retrieval recall, consider smaller chunks for dense facts, and expand synthetic evaluation cases.

16. File Map You Should Remember

File	Meaning
app/api.py	FastAPI routes and background ingestion jobs.
app/graph.py	LangGraph workflow and final answer routing.
app/retrieval/firewall.py	SafeRetriever and spoiler firewall.
app/retrieval/hybrid.py	Reciprocal Rank Fusion.
app/retrieval/oracle.py	Boolean future-answer oracle.
app/ingestion/chunker.py	Sentence-aware chunking with word offsets.
app/store/db.py	SQLite schema, chunks, sessions, jobs, conversations.
app/store/vectors.py	Embedding matrices and prefix slicing.
app/agents/validator.py	Citation and grounding validation.
app/agents/llm.py	LLM provider abstraction and key handling.

web/src/api.ts	Frontend API client.

17. Final 5-Minute Revision

- Say: agentic temporal RAG, not just chatbot.
- Explain: retrieval is limited by reader position.
- Mention: chunks have word_start and word_end.
- Say: eligible means word_end <= position.
- Vector safety: E[:n_p].
- Keyword safety: c.idx < n_p.
- LangGraph handles branches and retry loop.
- Answer agent must cite evidence or say INSUFFICIENT.
- Validator checks citations and grounding.
- Oracle sees future only as embeddings and returns only boolean.
- Main improvement: reranking and better retrieval recall.

Best closing statement:

> The strongest part of this project is that spoiler safety is not left to the LLM's obedience. The system controls the evidence before generation, validates the answer after generation, and uses graph routing to decide whether to answer, retry, refuse, or surface an error.